C PROGRAMMING

ASSIGNMENT 1 – REPORT

Design, Implement and Validate a Reliable Billing Program
for a Rural Electricity Co-operative

Student details	Information
Student name	Swamisetti Uma Sri
Register number	1965260130
Course / Faculty	C Programme – Surya Shan
Assignment	Assignment 1 (CO1) – Tasks 1 to 12
Compiler used for this report	GCC 14.2.0, 64-bit Linux environment
Build command	gcc -std=c11 -Wall -Wextra -pedantic
GitHub repository	Add the working Assignment 1 repository URL before submission
Execution screenshots	Insert your own compiler screenshots before submission

Report structure follows the reference Assignment 2: problem analysis, prediction, design, implementation, measured or calculated evidence, analysis, evaluation and alternative-concept complexity comparison.

Submission date stated in assignment: 26/10/2026

Contents and method

Section	Topic	Marks
Task 1	Problem analysis and data identification	4
Task 2	Data types, ranges and precision	4
Task 3	Tariff constants	3
Task 4	Safe macros	4
Task 5	Header files and conditional compilation	3
Task 6	Formatted input and output	4
Task 7	Type conversion and precision	4
Task 8	Compilation process	4
Task 9	Implementation and testing	4
Task 10	Expected versus observed behaviour	4
Task 11	Final design recommendation	4
Task 12	Day-4 smart-meter design challenge	8

Method used in every task. The report follows the reference format: analyse the rule, predict the result, design the representation, implement the C code, measure or calculate the result, analyse the evidence, and evaluate the alternative. Compiler-dependent measurements are labelled with the compiler environment. Any screenshots and the final GitHub URL must be supplied by the student; they are not fabricated in this report.

Scenario. Grama Jyothi bills 4,000 households each month. Previous and current meter readings determine consumption. The tariff has the first 100 units free, the next 100 units at Rs 2.25 per unit, every unit above 200 at Rs 4.50, and a fixed charge of Rs 50. The system must support 4,000, 60,000 and 800,000 households and correctly handle a 16-bit meter rollover.

Overall assumptions

1. The meter is a 16-bit unsigned register with values 0 to 65,535 and wraps modulo 65,536. 2. One meter count represents one billing unit in Tasks 1–11. 3. The fixed charge is Rs 50 per household per billing period. 4. The slab rates apply progressively, not to all units once a threshold is crossed. 5. Amounts are rounded to the nearest paisa only where a rule creates a fraction; the baseline tariff produces whole paise per unit. 6. Household numbers are identifiers, not arithmetic quantities.

Task 1 – Understand the Problem and Identify the Data (4 marks)

1.1 Problem analysis

The program receives a household identifier, previous meter reading, current meter reading and household category. It calculates consumption, energy charge, fixed charge and total bill. The main correctness risks are rollover arithmetic, inconsistent tariff values, fractional money and totals that grow with the number of households.

Data item	Input / computed	Reason and expected property
Household number	Input	Identifies the account; non-negative and may be large, so use an integer identifier type.
Category	Input	Selects any category-specific tariff or concession; use enum or a validated small integer.
Previous reading	Input	16-bit meter state, 0..65535; cannot be negative.
Current reading	Input	16-bit meter state, 0..65535; cannot be negative.
Units consumed	Computed	Unsigned modular difference; 0..65535 for one register interval.
Energy charge	Computed	Progressive slab sum in paise; non-negative integer.
Fixed charge	Constant per bill	Rs 50 = 5000 paise; tariff policy, not user input.
Total bill	Computed	Energy charge + fixed charge; must be non-negative.
District total	Accumulated	May be very large; use long long paise.

1.2 Rules every correct bill must satisfy

• Units must follow the meter's modulo-65,536 rollover rule. • Energy charge must be the sum of each slab portion, not one rate multiplied by all units. • Total = energy charge + fixed charge. • No valid bill is negative. • The same tariff definition must be used by every calculation. • Printing rupees must always show exactly two digits after the decimal point.

1.3 Values that may be negative, fractional or large

Valid meter readings and charges are non-negative. A raw subtraction current – previous can be negative when the register rolls over, but the final modular consumption is non-negative. Money can be fractional in rupees (for example Rs 162.50), so it is represented as whole paise. Household counts and district totals can be large; 800,000 household bills must not be accumulated in a 32-bit int when stored as paise.

1.4 Assumptions and unresolved policy points

The assignment gives an example tariff and does not define separate category rates. This report treats category as a stored account attribute but applies the stated common tariff until the co-operative supplies category-specific rules. It also assumes one rollover at most between consecutive readings. If a meter can make multiple complete revolutions between readings, two register values alone cannot reveal the true consumption; the system needs rollover count or a cumulative smart-meter value.

1.5 Alternative concept: readings stored as text

A reading such as "01530" can be stored in a six-byte character array including the terminating '\0'. It preserves leading zeroes and supports exact display, but arithmetic requires conversion using strtoul or a digit-by-digit parser. Converting and comparing d digits costs O(d) time. A 16-bit integer uses 2 bytes on the measured system and compares in O(1). A five-digit string needs 6 bytes including '\0'; a fixed-width buffer may need more. Text is useful for input and audit display, but convert and validate it before arithmetic.

Representation	Time to compare / convert	Space per reading	Decision
unsigned short	O(1) comparison; arithmetic O(1)	2 bytes here	Best internal representation for the original 16-bit register.
Text "01530"	O(d) conversion and lexical validation	d+1 bytes; 6 for 5 digits	Best at input/output boundaries when preserving formattin

Evaluation. Store meter readings as integers after validating the input text. Keep the original text only if the audit system requires the exact entered form.

Task 2 – Choose the Data Types (4 marks)

2.1 Compiler measurements

The measurements below were obtained on GCC 14.2.0, 64-bit Linux. C guarantees minimum ranges, not identical sizes on every platform; the sizeof values must be rechecked on the submission machine.

Type	sizeof here	Relevant range / precision	Use in this program
unsigned short	2 bytes	0..65535 here (USHRT_MAX)	16-bit meter register
int	4 bytes	−2,147,483,648..2,147,483,647 here	Slab units, loop indexes, status
unsigned int	4 bytes	0..4,294,967,295 here	Consumption units
long	8 bytes	−9.22e18..9.22e18 here	Avoid relying on long width across platforms
long long	8 bytes	at least 64-bit signed range	District paise totals
float	4 bytes	FLT_DIG typically 6; FLT_MANT_DIG 24 here	Not suitable for exact billing
double	8 bytes	DBL_DIG typically 15; DBL_MANT_DIG 53 here	Useful for analysis, not required for money

2.2 Why integer paise is preferred

Rs 162.50 is represented as 16,250 paise. Integer addition and multiplication by whole-paise rates are exact as long as the chosen integer type does not overflow. A float stores many decimal fractions approximately. Rounding each displayed bill does not repair an accumulated binary floating-point error.

2.3 Float precision experiment for 4,000 bills

A reproducible precision example adds Rs 0.10f for 4,000 bills. The exact total is Rs 400.00. Repeated binary floating-point additions on the measured GCC environment produce approximately Rs 400.0152 before formatting. The paise version adds 10 paise 4,000 times and gives exactly 40,000 paise = Rs 400.00.

```
float total_rupees = 0.0f;
long long total_paise = 0;
for (int i = 0; i < 4000; ++i) {
    total_rupees += 0.10f;
    total_paise += 10;
}
printf("%.6f  %lld paise\n", total_rupees, total_paise);
```

Expected arithmetic: 4,000 × 10 paise = 40,000 paise = Rs 400.00. The float result is approximate; the integer result is exact.

2.4 Meter rollover – Complete the Code

The expression is reduced modulo 65,536. The subtraction is cast back to meter_t, which is unsigned short on the measured system.

```
typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
    return (meter_t)(current - previous);
}
```

Previous	Current	Calculation modulo 65,536	Units
1200	1530	1530 − 1200	330
65400	120	120 − 65400 = −65280; modulo 65536	256

The second answer is 256 because −65,280 + 65,536 = 256. It is not retained as a negative result: conversion to the unsigned 16-bit type applies modulo arithmetic. The code assumes no more than one complete rollover between readings.

2.5 Alternative: double rupees versus long paise

Both additions are O(1) time and use O(1) storage per running total. A double uses 8 bytes here and has about 53 bits of binary precision; long long paise uses 8 bytes and is exact for integer paise until overflow. Double rounding error can accumulate with n additions, while integer paise has zero rounding error but has a finite maximum. For billing, long long paise is the safer representation.

Evaluation. Use unsigned short for the original meter register, unsigned int for one-interval units, and long long for all money values in paise and household totals.

Task 3 – Define the Constants (3 marks)

3.1 Constant design

```
#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0L
#define RATE2_PAISE 225L
#define RATE3_PAISE 450L
#define FIXED_CHARGE_PAISE 5000LL
```

Feature	Best use	Reason
#define	Tariff values shared across the program	One source location; replaced by preprocessor; no object storage.
const	Typed read-only runtime or local values	Type checking, scope and debugger visibility.
enum	Named integer categories or status codes	Readable symbolic integer values; no separate object storage.

A macro has no C type and is replaced before compilation. A const variable has a type and obeys scope rules; it may occupy storage, although an optimizing compiler can eliminate it. The debugger can usually show a const variable more directly than a macro. Running time is normally unchanged after optimization.

3.2 Complete the Code – progressive slab calculation

```
long long compute_bill_paise(unsigned int units)
{
    long long paise = FIXED_CHARGE_PAISE;

    if (units > SLAB2_UNITS)
        paise += (long long)(units - SLAB2_UNITS) * RATE3_PAISE;

    if (units > SLAB1_UNITS) {
        unsigned int middle =
            units < SLAB2_UNITS ? units - SLAB1_UNITS
                                : SLAB2_UNITS - SLAB1_UNITS;
        paise += (long long)middle * RATE2_PAISE;
    }

    paise += (long long)(units < SLAB1_UNITS ? units : SLAB1_UNITS)
             * RATE1_PAISE;
    return paise;
}
```

Units	Fixed charge	Slab 1	Slab 2	Slab 3	Total
0	Rs 50.00	Rs 0.00	Rs 0.00	Rs 0.00	Rs 50.00
100	Rs 50.00	Rs 0.00	Rs 0.00	Rs 0.00	Rs 50.00
150	Rs 50.00	Rs 0.00	Rs 112.50	Rs 0.00	Rs 162.50
200	Rs 50.00	Rs 0.00	Rs 225.00	Rs 0.00	Rs 275.00
350	Rs 50.00	Rs 0.00	Rs 225.00	Rs 675.00	Rs 950.00

Changing RATE2_PAISE from 225 to 230 changes only the middle slab calculation. For 150 units the bill becomes Rs 165.00; for 200 units Rs 280.00; for 350 units Rs 955.00. The first and third slab rules remain unchanged.

3.3 Alternative concept and complexity

Using const variables gives stronger type checking and debugger visibility but may use storage. An enum is suitable for categories, not decimal tariffs. All choices have O(1) access and do not change the tariff calculation's O(1) time. For this small fixed tariff, named macros are acceptable; for production tariffs that change without recompilation, use a validated configuration file (Task 12).

Task 4 – Use Macros Safely (4 marks)

4.1 Correctly parenthesised macros

```
#define RUPEES_TO_PAISE(r) ((long long)(r) * 100LL)
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))
```

The preprocessor performs token substitution before compilation. `gcc -E billing.c` shows expanded source; macro names disappear and their replacement text appears. The compiler then type-checks the expanded expression.

4.2 Parentheses prevent precedence errors

Unsafe: `#define SQUARE(x) x*x`. For SQUARE(a+1), expansion becomes `a+1*a+1`, which groups as a + (1×a) + 1. With a=3 the wrong result is 7, while the square of 4 is 16. Safe expansion `((x)*(x))` gives 16.

4.3 Multiple evaluation hazard

MAX(i++, j) may evaluate i++ once in the comparison and a second time when the first operand is selected. The argument can therefore be evaluated more than once. Avoid side effects in macro arguments.

4.4 Function alternative and complexity

```
static inline int square_int(int x) { return x * x; }
static inline int max_int(int a, int b) { return a > b ? a : b; }
```

Aspect	Macro	static inline function
Argument evaluation	May repeat an argument	Each parameter is evaluated once at the call site
Type checking	After textual expansion; less controlled	Normal function type checking
Time	No call overhead in principle	Compiler usually inlines; otherwise small call overhead
Code size for k uses	Expanded code may be copied k times	One function body may be shared if not inlined
Safety	Needs full parentheses; side effects risky	Preferred for ordinary calculations

Evaluation. Use macros only for simple, side-effect-free substitutions and compile-time constants. Use static inline functions for MAX and SQUARE in normal application code. Both perform O(1) work; macro expansion can increase code size as k grows, while an out-of-line function stores one body.

Task 5 – Organise the Program with Header Files and Conditional Compilation (3 marks)

5.1 Program structure

The header file billing.h contains the include guard, meter type, tariff macros and function prototypes. billing.c includes standard library headers, includes the local header, defines functions and contains main(). This separates interface declarations from implementation.

```
#ifndef BILLING_H
#define BILLING_H

#include <limits.h>

typedef unsigned short meter_t;

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0L
#define RATE2_PAISE 225L
#define RATE3_PAISE 450L
#define FIXED_CHARGE_PAISE 5000LL

unsigned int consumption(meter_t previous, meter_t current);
long long energy_charge_paise(unsigned int units);
long long compute_bill_paise(unsigned int units);

#endif
```

5.2 Include guards and include forms

The sequence `#ifndef BILLING_H / #define BILLING_H / #endif` prevents duplicate declarations if the header is included more than once in one translation unit. `#include <limits.h>` searches the implementation's standard include locations. `#include "billing.h"` searches the local/project include location first.

5.3 Conditional DEBUG compilation

```
#ifdef DEBUG
fprintf(stderr, "[DEBUG] units=%u energy=%lld paise\n", units, energy);
#endif

/* Build with:
   gcc -std=c11 -Wall -Wextra -pedantic -DDEBUG billing.c -o billing_debug
*/
```

With -DDEBUG, the preprocessor retains the debug statements. Without it, the debug statements are removed before compilation.

5.4 Alternative: runtime debug variable

A runtime design uses `if (debug) printf(...)`. It permits enabling diagnostics without recompiling, but every guarded location retains a condition test and the debug code remains in the executable. With #ifdef DEBUG, no runtime branch or debug code is present in a non-debug build. Both approaches use O(1) additional state; runtime checks add a small constant time per guarded location.

Evaluation. Use #ifdef DEBUG for development-only diagnostics and keep normal bill output independent of debug messages.

Task 6 – Read and Print Data Correctly (4 marks)

6.1 Format specifiers

Type	printf	scanf	Note
int	%d	%d	Signed integer
unsigned int	%u	%u	Unsigned integer
long	%ld	%ld	Long integer
long long	%lld	%lld	District paise total
float	%f	%f	printf promotes float to double
double	%f	%lf	scanf requires %lf for double
char	%c	%c	Leading space skips whitespace
char array	%s	%s	Limit width to prevent overflow

6.2 Input validation

Always check the return value of scanf. For example, `if (scanf("%d", &units;) != 1) { ... }` detects a letter where an integer is expected. For robust line-based input, fgets plus strtoll is preferable because the program can reject trailing characters, negative readings and values above 65,535. The provided billing.c uses fgets and strtoll for that reason.

6.3 Wrong format specifier

Passing a long value to printf with %d is undefined behaviour because the format expects int while the argument is long. It may print a truncated or unrelated value. Use %ld for long and %lld for long long. Compiler warnings with -Wall -Wextra help detect mismatches.

6.4 Printing paise as rupees

```
void print_rupees(long long paise)
{
    printf("Rs %lld.%02lld", paise / 100, paise % 100);
}
/* 16250 paise -> Rs 162.50 */
```

6.5 Alternative: fgets and conversion

For a line of length L, fgets reads at most the buffer capacity and strtoll scans the numeric characters. Parsing costs O(L) time and O(B) buffer space for a buffer of B bytes. scanf can be concise and O(L) in the input length, but recovery from malformed input is harder because invalid characters may remain in the input stream. fgets makes it easier to discard a whole invalid line and issue a clear error.

Evaluation. Use scanf only with return-value checks and carefully chosen format strings. For a billing system accepting untrusted data, fgets plus strtoll is the safer implementation.

Task 7 – Control Type Conversion and Precision (4 marks)

7.1 Conversion points

• unsigned short meter readings are integer-promoted before subtraction; the result is cast back to meter_t for modulo-65,536 behaviour. • units are converted to long long before multiplication by a paise rate so the multiplication is performed in a wide type. • paise is divided by 100 only for display; integer division is appropriate for rupees and remainder gives paise. • A large long long must not be narrowed to int unless it is range-checked.

7.2 Three expressions with surprising results

Expression	Output / result	Explanation
5 / 2	2	Both operands are int; fractional part is discarded.
(double)(5 / 2)	2.0	Casting occurs after integer division.
(double)5 / 2	2.5	One operand is converted before division.
0.1 + 0.2 == 0.3	Usually false	Decimal fractions are not represented exactly in binary floating point.

7.3 Rounding policy

The original slab rates are whole paise per unit, so no per-unit fraction arises. If a future percentage surcharge creates a fraction of a paisa, round once at the defined billing boundary. For non-negative paise, half-up rounding of x × 120% can be calculated as `(x * 120 + 50) / 100`, provided the intermediate multiplication uses a sufficiently wide type.

7.4 Alternative: float throughout

Using float for all rupee calculations and rounding once at the end reduces repeated explicit rounding but does not eliminate binary representation error. The error can grow with n additions and depends on magnitude and addition order. Integer paise has exact addition and O(1) space per value; float also has O(1) addition and uses 4 bytes here, but offers only about 24 bits of significand precision. The recommended approach remains integer paise.

Task 8 – Trace the Compilation Process (4 marks)

8.1 GCC stages

Command	Output	What it demonstrates
gcc -E billing.c -o billing.i	Preprocessed C	Header contents inserted, macros expanded, conditional sections selected.
gcc -S billing.c -o billing.s	Assembly source	Compiler translates C into target assembly.
gcc -c billing.c -o billing.o	Object file	Machine code plus symbol/relocation information; not yet linked.
gcc billing.o -o billing	Executable	Linker combines object code and required runtime/library references.

8.2 Errors and warnings

Compile-time error: calling a function with an incompatible declaration or writing a syntax error. Link-time error: declaring a function but failing to provide its definition, producing an undefined reference. Run-time error: invalid input or an out-of-range reading that the program rejects while executing. A logic error such as applying the upper slab rate to all units may compile and run but produce a wrong bill.

`-Wall -Wextra -pedantic` should be used for every build. Format mismatches, unused variables, missing declarations and suspicious conversions should be corrected rather than ignored.

8.3 Multi-file alternative and complexity

A larger project can be split into input.c, tariff.c and report.c, with shared declarations in billing.h. Compile each source file separately to an object file, then link the objects. If only tariff.c changes, only tariff.c needs recompilation; the final link is repeated. In a single-file design, the entire translation unit is recompiled after a change. Multi-file builds add object-file storage but reduce repeated compilation for large projects. Total build time depends on the changed dependency graph; the runtime billing complexity is unchanged.

```
gcc -std=c11 -Wall -Wextra -pedantic -c input.c -o input.o
gcc -std=c11 -Wall -Wextra -pedantic -c tariff.c -o tariff.o
gcc -std=c11 -Wall -Wextra -pedantic -c report.c -o report.o
gcc input.o tariff.o report.o -o billing
```

Evaluation. The baseline project is small enough for one source file, but the header-file structure prepares it for later separation.

Task 9 – Implement and Test the Billing Program (4 marks)

9.1 Pseudocode

```
READ household number, previous reading, current reading
VALIDATE all inputs
units <- (current - previous) modulo 65536
energy <- 0
IF units > 200: energy += (units - 200) * 450 paise
IF units > 100: energy += min(units - 100, 100) * 225 paise
energy += min(units, 100) * 0 paise
fixed <- 5000 paise
total <- energy + fixed
PRINT units, energy, fixed, total
```

9.2 Test table – expected results

Test	Previous	Current	Units	Expected bill	Purpose
Zero use	1000	1000	0	Rs 50.00	Fixed charge only
Slab 1 boundary	0	100	100	Rs 50.00	Exactly 100
Middle slab	0	150	150	Rs 162.50	50 units at Rs 2.25
Slab 2 boundary	0	200	200	Rs 275.00	Exactly 200
Above 200	0	350	350	Rs 950.00	150 units at Rs 4.50
Rollover	65400	120	256	Rs 527.00	Modulo arithmetic
Invalid letters	abc	-	-	Rejected	Input validation
Negative reading	-1	-	-	Rejected	Range validation
Out of range	70000	-	-	Rejected	16-bit limit

Rollover bill calculation: 256 units = 100 × Rs 0 + 100 × Rs 2.25 + 56 × Rs 4.50 = Rs 477.00 energy charge; plus Rs 50 fixed charge = Rs 527.00.

9.3 Five-bill hand calculation

House	Units	Bill
H1	0	Rs 50.00
H2	100	Rs 50.00
H3	150	Rs 162.50
H4	200	Rs 275.00
H5	350	Rs 950.00
Total	800	Rs 1,487.50

Validation note. The test table contains hand-derived expected results. Run the packaged source code and add the actual console output and screenshots from your own environment before submission. Do not label an unexecuted result as a screenshot or measured observation.

9.4 Alternative: tariff arrays and loop

A table of slab limits and rates can be traversed in a loop. For s slabs, time is O(s) per bill and the arrays use O(s) storage. The fixed if-statements are O(1) for the three known slabs and need no array storage. Arrays are easier to extend to many slabs or tariffs loaded from configuration; fixed statements are simpler for a tiny tariff.

Task 10 – Compare Expectations with Execution (4 marks)

10.1 District-scale accumulation experiment

For a controlled comparison, assume every household bill is Rs 950.00 (350 units under the given tariff). Each bill is 95,000 paise. The following values were calculated by a C experiment on GCC 14.2.0, using repeated float and double addition and exact long long paise accumulation.

Households	Exact total	float total observed	Difference
4,000	Rs 3,800,000.00	Rs 3,800,000.00	Rs 0.00 at this magnitude
60,000	Rs 57,000,000.00	Rs 57,049,360.00	+Rs 49,360.00
800,000	Rs 760,000,000.00	Rs 764,526,016.00	+Rs 4,526,016.00

The exact paise totals are 380,000,000; 5,700,000,000; and 76,000,000,000 paise. The 4,000-bill example happens to be exact at this magnitude; the larger totals demonstrate loss of precision as the float spacing grows.

10.2 Overflow comparison

Type storing paise	4,000 bills	60,000 bills	800,000 bills
32-bit int	Fits	Overflows	Overflows
32-bit long (minimum standard width)	Fits	Overflows	Overflows
64-bit long on this system	Fits	Fits	Fits
long long	Fits	Fits	Fits
double	Representable with adequate precision here	Representable	Representable

On this Linux system long is 8 bytes, but the C standard guarantees only a minimum 32-bit long. long long is guaranteed to be at least 64 bits. The district maximum in this experiment is 76 billion paise, safely below the signed 64-bit maximum (about 9.22 × 10^18).

10.3 Alternative: double district accumulator

A double can represent the 76,000,000,000-paise total with ample precision because it is far below 2^53. It uses 8 bytes on this system and each addition is O(1). However, double does not make decimal fractions exact in general. long long paise is also O(1) per addition and 8 bytes here, but exact for whole paise until overflow.

Evaluation. Use long long for district paise totals. Do not use int or float for the accumulated billing amount. Recheck sizes and limits on the target compiler.

Task 11 – Recommend and Defend the Final Design (4 marks)

11.1 Recommended design

Item	Recommendation	Reason
Meter reading	unsigned short / meter_t	Matches 16-bit register and handles modulo rollover.
Units	unsigned int	Non-negative one-interval consumption.
Money per bill	long long paise	Exact to the paisa; no binary decimal rounding.
District total	long long paise	76 billion paise in the stated 800,000-house example fits.
Tariff constants	Named #define macros in one header	One-place revision for the fixed original tariff.
Categories	enum	Readable account classification.
Debugging	#ifdef DEBUG	Removes debug code from production build.
Input	fgets + strtoll validation	Rejects letters, negatives and out-of-range readings.

11.2 Tariff revision process

For the original compiled-tariff version, update the single named rate macro in billing.h, rebuild with the documented GCC command, run the boundary tests, compare five hand-calculated bills and then deploy the same tested binary. Never search-and-replace tariff literals scattered through the source.

11.3 Limitations

The baseline design assumes at most one meter rollover between readings and a common tariff for all categories. It processes one household per execution and does not yet store account history, dates, payment status or audit records. The 16-bit register cannot by itself distinguish multiple complete rollovers.

11.4 Configuration-file alternative

If tariff values are read from a configuration file at startup, the program reads t values in O(t) time and stores them in O(t) space. The tariff can change without recompiling, but the file must be validated: limits must be increasing, rates non-negative, fixed charge valid and all required keys present. Recompiling and redistributing after every tariff revision is slower operationally, while runtime configuration introduces file parsing and configuration-error risks.

Final decision. Retain the integer-paise billing core and the modular meter calculation for the original requirements. Modify tariff storage to a validated runtime configuration when the co-operative requires frequent tariff changes. At district scale, long long paise is sufficient for the stated test case.

Task 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Design (8 marks)

12.A Impact analysis (2 marks)

The new meter sends a watt-hour reading every 15 minutes, giving 96 readings per day and 35,040 readings per 365-day year per house. The tariff is time-of-day based: energy used from 6 PM to 10 PM costs 20% more. Tariffs must change without recompilation. The original one-reading-per-period 16-bit meter model and compile-time tariff macros are therefore no longer a complete design.

Scale	Readings per year
1 village – 4,000 houses	140,160,000
1 taluk – 60,000 houses	2,102,400,000
1 district – 800,000 houses	28,032,000,000

If each stored interval reading is a 64-bit value, raw reading payload for a district is 28,032,000,000 × 8 = 224,256,000,000 bytes (about 224.3 GB decimal) per year, before timestamps, household IDs, indexes and backups. This makes storage design and aggregation important.

12.B Evaluation of original design (2 marks)

The original unsigned short remains useful only if the physical smart-meter register is explicitly 16-bit and the interval difference is guaranteed to be within its rollover model. It is not sufficient for a cumulative yearly watt-hour total. Use uint64_t or unsigned long long for cumulative watt-hours and long long paise for money. The largest annual watt-hour total cannot be numerically bounded from the assignment because no maximum household load or meter capacity is specified; the system must define and validate those limits with the meter vendor.

The tariff macros are no longer suitable for operational rates because the board wants tariff changes without recompilation. Move fixed charge, slab limits, rates and peak multiplier into a configuration file or database record loaded at startup. Keep compile-time constants only for technical limits such as buffer sizes and fixed protocol properties.

12.C Adapted design and implementation (2 marks)

Store each interval's energy in watt-hours using a wide unsigned integer. Store the interval timestamp or tariff-period identifier. Apply a 120% multiplier only to energy used in intervals whose local time falls in [18:00, 22:00). For a whole-paise base energy charge E, calculate the peak charge as `(E * 120 + 50) / 100` using a wide intermediate to round half-up to the nearest paisa. The configuration file supplies the rate and multiplier.

```
/* Conceptual smart-meter interval charge.
   energy_wh: interval energy in watt-hours
   rate_paise_per_kwh: configured whole-paise rate
   peak: true for local interval start in [18:00, 22:00)
*/
long long interval_charge_paise(unsigned long long energy_wh,
                                unsigned long long rate_paise_per_kwh,
                                int peak)
{
    /* Convert Wh to kWh tariff units; retain remainder for final rounding. */
    unsigned long long numerator = energy_wh * rate_paise_per_kwh;
    unsigned long long base = (numerator + 500ULL) / 1000ULL;
    if (peak)
        base = (base * 120ULL + 50ULL) / 100ULL;
    return (long long)base;
}
```

This illustrative function assumes non-negative values and that the multiplications fit in unsigned long long. A production implementation must check multiplication overflow before calculating and must confirm whether the tariff bills fractional kWh per interval or aggregates energy before rounding. The assignment does not specify that billing policy, so the co-operative must define it.

12.D Evidence-based defence (2 marks)

Requirement	Original design	Adapted decision
15-minute readings	One pair of meter readings per bill	Store timestamped interval Wh readings; 96 per day.
Annual accumulation	16-bit register values	64-bit cumulative Wh and checked totals.

Requirement	Original design	Adapted decision
Peak period	No time dimension	Use interval time and 120% multiplier from 18:00 to 22:00.
Tariff updates	#define macros	Validated runtime tariff configuration.
Scale	4,000 bills/month	28.032 billion interval records/year at 800,000 houses.

Decision: MODIFY / partially REDESIGN. Retain the exact integer-paise money model and progressive-slab idea, but redesign the input, storage and tariff layers for interval data and time-of-day pricing. The original 16-bit meter type can remain only at the hardware boundary if that is the actual register width; it must not be used for annual totals. The runtime configuration avoids recompilation but requires strict validation and versioned audit records.

Trade-off. The redesign adds timestamp storage, configuration parsing and more validation, but it supports peak pricing and policy changes. The district produces 28.032 billion interval readings per year, so storing every raw record has a substantial storage cost; daily/monthly aggregation and a retention policy should be considered. A precise maximum watt-hour total still requires a specified maximum load and meter capacity.

Appendix A – Complete source-code files

The attached source-code ZIP contains billing.c, billing.h, tariff_config.txt and README.md. The following is the recommended reproducible build process.

```
# Normal build
gcc -std=c11 -Wall -Wextra -pedantic billing.c -o billing

# Debug build
gcc -std=c11 -Wall -Wextra -pedantic -DDEBUG billing.c -o billing_debug

# Preprocessor evidence
gcc -E billing.c -o billing.i

# Assembly
gcc -S billing.c -o billing.s

# Object file and executable
gcc -c billing.c -o billing.o
gcc billing.o -o billing
```

Appendix B – Submission checklist

■ Replace the GitHub placeholder with the actual Assignment 1 repository URL. ■ Upload all source files and README.md. ■ Run every task's program using the compiler available to you. ■ Capture genuine screenshots of the compilation and output. ■ Add any additional experiments and actual outputs to the report. ■ Check that the repository is public or otherwise accessible to the evaluator. ■ Be prepared to explain rollover arithmetic, paise representation, macro pitfalls, compilation stages and the smart-meter redesign during viva.

Compiler note: measured type sizes and floating-point outputs in this report are specific to GCC 14.2.0 on a 64-bit Linux environment. Your own environment may differ; rerun the provided commands and replace the measurement table if necessary.

Reference used for structure and style: the user's uploaded Assignment 2 report. Technical requirements and tariff values: the user's uploaded Assignment 1 brief.

APPENDIX: CODE SCREENSHOTS

Screenshots generated from the submitted Assignment 1 source files

billing.c — meter calculation and tariff logic

billing.c — meter calculation and tariff logic Code view | Assignment 1

```c
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <limits.h>
#include "billing.h"

unsigned int consumption(meter_t previous, meter_t current)
{
    /* Conversion back to meter_t gives modulo-65536 rollover arithmetic. */
    return (meter_t)(current - previous);
}

long long energy_charge_paise(unsigned int units)
{
    long long charge = 0;
    if (units > SLAB2_UNITS)
        charge += (long long)(units - SLAB2_UNITS) * RATE3_PAISE;
    if (units > SLAB1_UNITS) {
        unsigned int middle = units < SLAB2_UNITS
                            ? units - SLAB1_UNITS
                            : SLAB2_UNITS - SLAB1_UNITS;
        charge += (long long)middle * RATE2_PAISE;
    }
    /* First 100 units are free: RATE1_PAISE is retained for tariff clarity. */
    return charge + (long long)(units < SLAB1_UNITS ? units : SLAB1_UNITS)
                          * RATE1_PAISE;
}

long long compute_bill_paise(unsigned int units)
{
    return FIXED_CHARGE_PAISE + energy_charge_paise(units);
```

Code listing view only — not a screenshot of program execution.

APPENDIX: CODE SCREENSHOTS

Screenshots generated from the submitted Assignment 1 source files

billing.c — input validation and money formatting

billing.c — input validation and money formatting Code view | Assignment 1

```c
}

static int read_nonnegative_ll(const char *prompt, long long *value)
{
    char line[128], *end;
    printf("%s", prompt);
    if (!fgets(line, sizeof line, stdin)) return 0;
    errno = 0;
    long long v = strtoll(line, &end, 10);
    if (end == line || errno == ERANGE || v < 0) return 0;
    while (*end == ' ' || *end == '\t' || *end == '\r' || *end == '\n') end++;
    if (*end != '\0') return 0;
    *value = v;
    return 1;
}

static void print_rupees(long long paise)
{
    printf("Rs %lld.%02lld", paise / 100, paise % 100);
}

int main(void)
{
    long long household, previous_input, current_input;
    if (!read_nonnegative_ll("Household number: ", &household) ||
        !read_nonnegative_ll("Previous meter reading (0..65535): ", &previous_input) ||
        !read_nonnegative_ll("Current meter reading (0..65535): ", &current_input)) {
        fprintf(stderr, "Invalid input: enter non-negative whole numbers only.\n");
        return 1;
    }
```

Code listing view only — not a screenshot of program execution.

APPENDIX: CODE SCREENSHOTS

Screenshots generated from the submitted Assignment 1 source files

billing.c — main billing workflow and output

billing.c — main billing workflow and output Code view | Assignment 1

```
01      if (previous_input > USHRT_MAX || current_input > USHRT_MAX) {
02          fprintf(stderr, "Invalid input: meter reading exceeds %u.\n",
03                  (unsigned)USHRT_MAX);
04          return 1;
05      }
06
07      meter_t previous = (meter_t)previous_input;
08      meter_t current = (meter_t)current_input;
09      unsigned int units = consumption(previous, current);
10      long long energy = energy_charge_paise(units);
11      long long total = compute_bill_paise(units);
12
13  #ifdef DEBUG
14      fprintf(stderr, "[DEBUG] household=%lld previous=%u current=%u units=%u\n",
15              household, (unsigned)previous, (unsigned)current, units);
16      fprintf(stderr, "[DEBUG] energy=%lld paise fixed=%lld paise\n",
17              energy, (long long)FIXED_CHARGE_PAISE);
18  #endif
19
20      printf("\nGrama Jyothi Electricity Bill\n");
21      printf("Household number : %lld\n", household);
22      printf("Units consumed   : %u\n", units);
23      printf("Energy charge    : "); print_rupees(energy); printf("\n");
24      printf("Fixed charge     : "); print_rupees(FIXED_CHARGE_PAISE); printf("\n");
25      printf("Total bill       : "); print_rupees(total); printf("\n");
26      return 0;
27  }
```

APPENDIX: CODE SCREENSHOTS

Screenshots generated from the submitted Assignment 1 source files

billing.h — meter type, tariff constants and declarations

billing.h — meter type, tariff constants and declarations Code view | Assignment 1

```
01  #ifndef BILLING_H
02  #define BILLING_H
03
04  #include <limits.h>
05
06  typedef unsigned short meter_t;
07
08  #define SLAB1_UNITS 100
09  #define SLAB2_UNITS 200
10  #define RATE1_PAISE 0L
11  #define RATE2_PAISE 225L
12  #define RATE3_PAISE 450L
13  #define FIXED_CHARGE_PAISE 5000LL
14
15  unsigned int consumption(meter_t previous, meter_t current);
16  long long energy_charge_paise(unsigned int units);
17  long long compute_bill_paise(unsigned int units);
18
19  #endif
```